From Branches to Bytes: A Negative Result in Extending Learned Static Prediction to Data Layout Optimization

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

Machine learning has shown promise in compiler optimization, with learned static prediction achieving profile-guided optimization benefits for branch probabilities without runtime profiling. We investigate whether this methodology extends to data layout optimization, a harder problem characterized by higher-dimensional outputs, context-dependent optimality, and legality constraints. We present LayoutLearner, an XGBoost-based system trained on 60 samples from 20 benchmarks to predict profitable structure layout transformations. Our key finding is that learned static prediction achieves an F1 score of 0.690, significantly underperforming simple static heuristics (F1=0.909) that require no training. The model exhibits poor discriminative ability (ROC-AUC=0.367) and introduces 54.5% compilation-time overhead. While we refute our original hypothesis that ML would approach profile-guided performance, we identify fundamental challenges—limited training data, feature engineering difficulties, and the inherent complexity of layout decisions—that explain this gap. We frame our results as establishing baseline challenges for the community and suggest hybrid approaches and improved feature extraction as promising directions.

1 Introduction

Data layout optimizations, such as structure splitting and field reordering, are critical for improving cache performance in modern applications. By reorganizing how data structures are laid out in memory, compilers can significantly reduce cache misses and improve spatial locality. However, identifying profitable data layout transformations is challenging because access patterns vary by code region, static analysis cannot reliably predict runtime behavior, and profile-guided optimization (PGO) requires expensive profiling runs with representative input data.

Current production compilers have largely abandoned automatic structure layout optimization. GCC's `-fipa-struct-reorg` was removed in version 4.8 due to correctness issues and lack of link-time optimization support. This leaves a significant performance opportunity untapped.

Rotem and Cummins (2021) demonstrated that lightweight machine learning models (XGBoost) trained on static LLVM IR features can effectively predict branch probabilities, achieving PGO benefits without profiling overhead. Their work establishes that "profile-guided optimization without profiles" is viable for control-flow decisions. Our key insight was that this proven methodology could extend to data layout optimization.

However, data layout optimization presents challenges that make it inherently more difficult than branch prediction: (1) **higher-dimensional output space**—branch prediction is binary while layout requires predicting field groupings and ordering; (2) **context-dependent optimality**—layout quality depends on surrounding code regions and aliasing relationships; (3) **legality constraints**—layout errors affect correctness, not just performance; (4) **delayed feedback**—cache effects are harder to measure than branch outcomes; and (5) **transformation synthesis**—layout requires generating IR modifications, not just metadata.

1.1 Contributions

Our contributions are:

- We present LayoutLearner, the first application of learned static prediction to data layout optimization, extending the methodology of Rotem and Cummins (2021) to a new domain.
- We conduct a rigorous empirical evaluation on 60 samples from 20 benchmarks, comparing against strong static heuristics and profile-guided upper bounds.
- We provide an honest negative result: learned prediction (F1=0.690) underperforms simple heuristics (F1=0.909), with ROC-AUC (0.367) worse than random guessing.
- We identify fundamental challenges—dataset scale, feature engineering limitations, and the inherent complexity of layout decisions—that explain the performance gap and suggest directions for future work.

2 Related Work

2.1 Learned Static Prediction

Rotem and Cummins (2021) demonstrated "Profile Guided Optimization without Profiles" using XGBoost models trained on LLVM IR features to predict branch probabilities. Their approach achieves 75% prediction accuracy for branch weight classes and a 1.6% geomean speedup over non-PGO compilation. This work establishes the viability of learned static prediction and provides our methodological foundation. We apply their XGBoost-on-LLVM-IR methodology to data layout optimization.

2.2 Static Heuristics for Compiler Optimization

Ball and Larus (1993) introduced influential static heuristics for branch prediction (loop header, pointer comparison, etc.), achieving approximately 80% accuracy. Wu and Larus (1994) extended this to static branch frequency estimation. Calder et al. (1997) improved prediction using decision trees trained on program features. These works target control-flow prediction; we adapt similar heuristics for data layout decisions.

2.3 Data Layout Optimization

Profile-guided approaches use runtime information to guide layout decisions. Chilimbi et al. (1999) introduced cache-conscious structure definition using profiling to identify hot fields. Zhong et al. (2004) used whole-program reference affinity from profile data for array regrouping and structure splitting. These approaches require runtime profiling; we eliminate profiling through static prediction.

Rohwedder et al. (2024) introduced RebaseDL, a region-based structure splitting system using static data reuse analysis without profiling. RebaseDL identifies code regions with poor cache utilization and applies transformations, achieving up to 1.34× speedups. The key distinction is that RebaseDL uses *static analysis* with manually-tuned thresholds, while our approach uses *machine learning* to automatically learn prediction rules. We view these approaches as complementary—ML may identify opportunities that conservative static analysis misses, while static analysis provides correctness guarantees.

2.4 Machine Learning in Compilers

MLGO (Trofin et al., 2021) uses reinforcement learning for inlining and register allocation. CompilerGym (Cummins et al., 2021) provides environments for ML-based compiler optimization research. LOOPer (Merouani et al., 2024) applies deep learning to polyhedral loop

optimization. These works target different optimization problems; no prior work applies learned static prediction specifically to data layout optimization.

3 Method

3.1 Overview

LayoutLearner extends Rotem and Cummins's methodology to data layout optimization:

1. **Feature extraction**: Extract static features from LLVM IR representing structure access patterns.
2. **Model training**: Train lightweight ML models (XGBoost) to predict profitable layout transformations.
3. **Transformation prediction**: Identify split/reorder candidates and expected profitability.

Unlike branch prediction, data layout requires memory-access-focused features rather than control-flow features.

3.2 Feature Engineering

We extract 25 features across three categories:

Structural Features (8): Number of fields, total structure size, average/max/min field size, size variance, number of pointer fields, number of primitive fields.

Access Pattern Features (10): Average/max loop nesting depth at access sites, total access sites, average accesses per field, max accesses for any field, access variance, hot/cold field ratio, co-occurrence score, pointer arithmetic presence.

Context Features (7): Estimated function hotness using Wu and Larus-style heuristics, trip count estimates, allocation-in-loop indicator, dominance depth, kernel function indicator, benchmark type indicators (linear algebra, stencil, synthetic).

3.3 Model Architecture

Following Rotem and Cummins's proven approach, we use Gradient Boosted Decision Trees (XGBoost) as our primary model. This choice provides: (1) fast inference ($\sim$microseconds per prediction); (2) interpretable feature importance; (3) robustness to feature scaling issues; and (4) deployability within LLVM.

The prediction target is binary: whether splitting or reordering is profitable for a given structure and code region. We use the following hyperparameters: $n_estimators = 100$, $max_depth = 6$, $learning_rate = 0.1$, $objective =$ binary:logistic.

3.4 Training Methodology

We compile benchmarks to LLVM bitcode at `-O0`, extract features, and label optimal transformations based on manual analysis of access patterns. Models are trained with 5-fold cross-validation and evaluated on held-out test benchmarks using three random seeds (42, 123, 456).

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on 60 samples from 20 benchmarks: 14 PolyBench kernels (linear algebra and stencil computations), 6 synthetic microbenchmarks with known optimal layouts. Each benchmark has 3 data size variants (SMALL, MEDIUM, LARGE).

Table 1: Prediction accuracy comparison. Best non-oracle results in **bold**. ↑ indicates higher is better. Standard deviation shown across 3 seeds.

Method	Precision ↑	Recall ↑	F1-Score ↑	ROC-AUC ↑
Static Heuristics	**0.833 ± 0.000**	**1.000 ± 0.000**	**0.909 ± 0.000**	—
LayoutLearner	0.714 ± 0.000	0.667 ± 0.000	0.690 ± 0.000	0.367 ± 0.000
Profile-Guided (Oracle)	1.000	1.000	1.000	1.000

Table 2: Feature ablation study showing F1-Score when removing each feature category.

Ablation	F1-Score
Full Model	0.690
Without Access Pattern	0.593 (-14.1%)
Without Context	0.667 (-3.3%)
Without Structural	0.714 (+3.6%)

Baselines. We compare against:

- **Static Heuristics**: Three hand-crafted heuristics—(1) fields in deeply nested loops are hot, (2) fields with frequent accesses are hot, (3) fields accessed together should be grouped—combined via weighted voting.
- **Profile-Guided Oracle**: Using ground truth labels as predictions, establishing an upper bound.

Metrics. We report Precision, Recall, F1-Score, ROC-AUC, and inference time. Results are averaged across three random seeds with standard deviations.

Implementation. We use XGBoost 2.0.3, scikit-learn 1.4.0, and llvmlite 0.41.1 for bitcode processing. Experiments run on CPU only.

4.2 Main Results

Table 1 presents our main results. The key finding is that LayoutLearner (F1=0.690) significantly underperforms simple static heuristics (F1=0.909). Notably, the heuristics require no training data and achieve near-perfect recall (1.000), suggesting they rarely miss profitable transformations.

The ROC-AUC of 0.367 is particularly concerning—it is worse than random guessing (0.500), indicating the model has poor discriminative ability. The zero standard deviation across seeds suggests deterministic behavior due to the small dataset size and limited randomness in XGBoost's training procedure.

4.3 Ablation Studies

Table 2 shows the contribution of each feature category. Access pattern features contribute most—removing them causes a 14.1% performance drop. Surprisingly, removing structural features improves performance (+3.6%), suggesting these features may introduce noise or the model fails to use them effectively.

Table 3 shows that increasing model complexity does not improve performance—a simpler XGBoost with 50 trees achieves the best F1 (0.714), while deeper models overfit. All variants have inference times well under 1ms, satisfying the compilation-time constraint.

4.4 Cross-Benchmark Generalization

Table 4 reveals generalization challenges. Models trained on synthetic benchmarks perform poorly on linear algebra (F1=0.417), while those trained on stencil benchmarks generalize

Table 3: Model complexity ablation showing accuracy vs. inference time trade-off.

Model	F1-Score ↑	Inference Time (μs) ↓
Decision Tree	0.650	147
XGBoost (50 trees, depth 4)	**0.714**	380
XGBoost (100 trees, depth 6)	0.690	380
XGBoost (200 trees, depth 10)	0.690	383

Table 4: Cross-benchmark generalization: training on one benchmark type, testing on another.

Training Set	Test Set	F1-Score
Linear Algebra	Stencil	0.727
Linear Algebra	Synthetic	0.500
Stencil	Linear Algebra	0.744
Stencil	Synthetic	0.593
Synthetic	Linear Algebra	0.417
Synthetic	Stencil	0.737

reasonably to linear algebra (F1=0.744). This suggests domain-specific patterns do not transfer well.

4.5 Compilation-Time Overhead

Feature extraction takes 31.9ms and model inference takes 22.7ms, for a total overhead of 54.5% relative to baseline compilation (100ms). This far exceeds our 5% target. The overhead is dominated by Python-based feature extraction from bitcode; a C++ implementation within LLVM would reduce this significantly.

4.6 Success Criteria Validation

We validate against our original hypothesis:

1. **Within 20% of profile-guided**: FAIL. LayoutLearner F1 (0.690) vs. target (0.800).
2. **Compilation overhead $<$ 5%**: FAIL. Actual overhead is 54.5%.
3. **Identifies profitable on $\geq$40%**: PASS. Accuracy is 52.6%.
4. **Outperforms static heuristics**: FAIL. Heuristics F1 (0.909) vs. LayoutLearner (0.690).

The hypothesis is **REFUTED** (1 of 4 criteria passed).

5 Discussion

5.1 Why ML Underperforms Heuristics

Our central question: why does machine learning underperform simple heuristics on this task? We identify three factors:

Limited training data. With only 60 samples, the model cannot learn robust patterns. Static heuristics encode decades of compiler engineering knowledge; our model must learn equivalent rules from scratch with limited data.

Feature engineering challenges. Our static features (loop nesting, access counts) are indirect proxies for cache behavior. Heuristics directly encode expert knowledge about which

patterns matter. The negative contribution of structural features suggests our feature set is suboptimal.

The inherent complexity of layout decisions. Unlike branch prediction (binary, localized), layout decisions involve: (1) predicting which fields to group, (2) determining optimal ordering, (3) assessing profitability across different code regions. This higher-dimensional output space is harder to learn.

5.2 When Might ML Win?

Despite the negative result, we identify scenarios where ML could outperform heuristics:

- **Non-obvious patterns**: Heuristics may miss complex interactions between multiple features that ML could learn with sufficient data.
- **New architectures**: ML could adapt to novel cache hierarchies where hand-tuned heuristics fail.
- **Hybrid approaches**: ML could refine heuristic predictions, handling edge cases that heuristics miss.

5.3 On the Low ROC-AUC

The ROC-AUC of 0.367 (worse than random) indicates the model produces poorly calibrated probability estimates. This may stem from: (1) class imbalance in the training data, (2) the small dataset size preventing proper probability calibration, or (3) the model learning spurious correlations that do not generalize. We recommend future work explore calibration techniques and larger datasets.

5.4 Limitations

Our study has important limitations:

- **Small dataset**: 60 samples is insufficient for robust ML. Results may not generalize.
- **Simulation-only**: No actual LLVM implementation; evaluation uses simulated predictions.
- **Synthetic benchmarks**: Limited diversity compared to real-world codebases.
- **No runtime performance**: We measure prediction accuracy, not actual cache miss reduction or speedup.

5.5 Implications for the Community

We frame this work as establishing baseline challenges for ML-based data layout optimization:

1. Dataset construction for layout optimization is non-trivial and requires community effort.
2. Feature engineering for cache behavior prediction needs improvement—current static features are insufficient.
3. Simple heuristics are surprisingly strong baselines; future work must justify ML complexity.
4. The gap between static prediction and profile-guided optimization remains significant for data layout.

6 Conclusion

We investigated whether learned static prediction, successful for branch probabilities, extends to data layout optimization. Our evaluation of LayoutLearner on 60 samples from 20 benchmarks reveals that machine learning (F1=0.690) underperforms simple static heuristics (F1=0.909) and fails to approach profile-guided performance.

While we refute our original hypothesis, we contribute: (1) the first application of learned static prediction to data layout, (2) empirical evidence of the challenges involved, and (3) identification of directions for future work. We recommend: larger datasets with thousands of samples, improved feature engineering capturing cache behavior more directly, and hybrid approaches combining heuristics with learned refinements.

Data layout optimization remains a challenging domain where decades of compiler engineering knowledge, encoded in simple heuristics, outperforms current machine learning approaches. We hope this negative result guides the community toward more promising approaches and realistic expectations for ML in compiler optimization.

References

Thomas Ball and James R. Larus. Branch prediction for free. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation (PLDI)*, pages 300–313, 1993.

Brad Calder, Dirk Grunwald, Michael Jones, Donald Lindsay, James Martin, Michael Mozer, and Benjamin Zorn. Evidence-based static branch prediction using machine learning. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 19(1):188–222, 1997.

Trishul M. Chilimbi, Bob Davidson, and James R. Larus. Cache-conscious structure definition. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, pages 13–24, 1999.

Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, Yuandong Tian, and Hugh Leather. CompilerGym: Robust, performant compiler optimization environments for AI research. *arXiv preprint arXiv:2109.08267*, 2021.

Massinissa Merouani, Xujie Si, and Jose Nelson Amaral. LOOPer: A learned automatic code optimizer for polyhedral compilers. *arXiv preprint arXiv:2405.17712*, 2024.

Caio S. Rohwedder, Joao P. L. De Carvalho, and J. Nelson Amaral. Region-based data layout via data reuse analysis. In *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction (CC)*, pages 1–11, 2024.

Nadav Rotem and Chris Cummins. Profile guided optimization without profiles: A machine learning approach. *arXiv preprint arXiv:2112.14679*, 2021.

Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. MLGO: a machine learning guided compiler optimizations framework. *arXiv preprint arXiv:2101.04808*, 2021.

Youfeng Wu and James R. Larus. Static branch frequency and program profile analysis. In *Proceedings of the 27th Annual ACM/IEEE International Symposium on Microarchitecture (MICRO)*, pages 1–11, 1994.

Yutao Zhong, Maksim Orlovich, Xipeng Shen, and Chen Ding. Array regrouping and structure splitting using whole-program reference affinity. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, pages 255–266, 2004.